SWISSMICROS DM42

MATRICES
& LINEAR ALGEBRA

HANDBOOK

Theory, keystrokes, diagnostics, and engineering applications

Verified against the current DM42 manual and implementation

About this handbook

This book joins three layers that are too often separated: the mathematics, the exact DM42 workflow, and the numerical judgement needed to trust a result. It covers real and complex matrices, vectors, linear systems, transformations, applied models, and advanced constructions that can be assembled from the calculator's native primitives.

The research baseline is the SwissMicros DM42 User Manual v3.42 (24 September 2026), the official DM42 function reference, the DM42/Free42 implementation current at the date above, and established numerical linear algebra texts. Menu wording can move between firmware releases; the mathematical behavior and command semantics stated here were checked against source.

The XEQ key in program mode

In normal calculator mode, XEQ starts execution of a named function or program. In PRGM mode, pressing XEQ *inserts an* `XEQ` *instruction on the current program line*; it does not execute that instruction while you are entering the program. Every program-entry sequence in this handbook uses that distinction.

Notation and responsibility

Rows and columns are numbered from 1 in explanations and in the DM42 interface. Stack order is written $Y:X$, meaning the left operand is in Y and the right operand is in X. Examples are educational; independently validate safety-critical engineering, financial, navigation, or scientific work.

Contents at a glance

I Orientation and mathematical foundations 1

1 Why matrices belong on the DM42 2

1.1 The calculator's numerical model . . . 2
1.2 Native strengths and deliberate limits . . . 2
1.3 RPN orientation . . . 3
1.4 Command entry and program entry . . . 3

2 Matrix foundations 5

2.1 Shape, entries, and indexing . . . 5
2.2 Important matrix families . . . 5
2.3 Equality, addition, and scalar multiplication . . . 6
2.4 Matrix multiplication . . . 6
2.5 Transpose and conjugate transpose . . . 7
2.6 Determinant, inverse, rank, and linear independence . . . 7

3 Vectors: geometry carried in matrices 8

3.1 Column vectors and the matrix view . . . 8
3.2 Length, direction, and projection . . . 8
3.3 The cross product . . . 9
3.4 Torque and work . . . 9
3.5 Chapter review . . . 10

II The DM42 matrix workspace 11

4 Creating and naming matrices 12

4.1 The Matrix menu . . . 12
4.2 Create a matrix on the stack . . . 12
4.3 Create or resize a named matrix . . . 13
4.3.1 What resize preserves . . . 13
4.4 Inspect shape . . . 13

5 The matrix editor **14**
5.1 Entering and leaving edit mode . . . 14
5.2 Editing a 3×3 example . . . 14
5.3 Complex matrices . . . 15
5.4 Strings in real matrices . . . 15

6 Indexed access and structural editing **16**
6.1 Select an indexed matrix . . . 16
6.2 Set and recall I, J . . . 16
6.3 Element operations . . . 16
6.4 Extract and insert submatrices . . . 17
6.5 Insert, delete, and swap rows . . . 17
6.6 Safe structural workflow . . . 18

7 Storage, stack, and program patterns **19**
7.1 Named matrices versus stack matrices . . . 19
7.2 The four-level and big stack . . . 19
7.3 A reusable matrix program . . . 19
7.4 Dimension guard pattern . . . 20
7.5 Programmatic element traversal . . . 20
7.6 Checkpoint project: build and inspect . . . 21

III Core operations and geometric meaning **22**

8 Arithmetic on the RPN stack **23**
8.1 Addition and subtraction . . . 23
8.2 Scalar multiplication and division . . . 23
8.3 Matrix multiplication . . . 23
8.4 Powers . . . 24

9 Linear transformations **25**
9.1 A matrix as a machine . . . 25
9.1.1 Two-dimensional primitives . . . 25
9.2 Composition order . . . 25
9.3 Homogeneous coordinates . . . 26

10 Determinants, inverses, and division **27**
10.1 Determinants . . . 27
10.2 Inverse . . . 27
10.3 Why solving beats forming an inverse . . . 27
10.4 Multiple right-hand sides . . . 28

11 Norms, sums, and complex matrices **29**
11.1 Frobenius and row norms . . . 29
11.2 Row sums . . . 29
11.3 Complex representation . . . 29
11.3.1 Transpose is not adjoint . . . 29
11.3.2 Complex dot product . . . 30
11.4 Conjugation construction . . . 30
11.5 Norm-based checks . . . 30

IV Linear systems and numerical reliability **32**

12 Solving square systems **33**
12.1 The model $Ax = b$ . . . 33
12.2 The SIMQ application . . . 33
12.3 Worked 2×2 system . . . 33
12.4 The programmable solve . . . 34
12.5 Complex systems . . . 34

13 Unique, inconsistent, and underdetermined systems **35**
13.1 Geometric classification . . . 35
13.2 Two diagnostic examples . . . 35
13.2.1 Inconsistent . . . 35
13.2.2 Infinitely many . . . 36
13.3 Rectangular systems . . . 36

14 Conditioning, precision, and diagnostics **37**
14.1 Residual versus forward error . . . 37
14.2 A transparent ill-conditioned example . . . 37
14.3 A practical diagnostic ladder . . . 38
14.4 Scaling . . . 38
14.5 Error messages as evidence . . . 38

15 Least squares and fitting **39**
15.1 The normal-equation construction . . . 39
15.2 Line fit example . . . 39
15.3 Centering and scaling a fit . . . 40
15.4 Underdetermined minimum norm . . . 40

V Applied models **42**

16 Networks in engineering and physics **43**

16.1 DC nodal analysis . . . 43
16.2 AC phasor analysis . . . 43
16.3 Thermal resistance network . . . 44
16.4 Statics and force balance . . . 44
16.5 Rigid-body rotation . . . 44

17 Chemistry, economics, and decisions **46**

17.1 Balancing a chemical equation . . . 46
17.2 Portfolio mean and variance . . . 46
17.3 A two-strategy zero-sum game . . . 47
17.4 Leontief input–output model . . . 47
17.5 Sensitivity from repeated solves . . . 47

18 Markov chains, coordinates, and data fusion **48**

18.1 Markov transition matrices . . . 48
18.2 Stationary distribution . . . 48
18.3 Astronomical coordinate rotation . . . 49
18.4 Sensor fusion by weighted least squares . . . 49
18.5 Logistics and flow balance . . . 49
18.6 Application handoff checklist . . . 50

VI Advanced constructions **51**

19 Factorizations and eigenproblems **52**

19.1 LU factorization: present internally, not exposed generally . . . 52
19.2 Cholesky, QR, and SVD boundaries . . . 52
19.3 Analytic 2×2 eigenvalues . . . 52
19.4 Power iteration . . . 53
19.5 Rayleigh quotient . . . 53

20 Iterative and derived matrix methods **54**

20.1 Stationary iteration . . . 54
20.2 Matrix exponential . . . 54
20.3 Cayley–Hamilton shortcuts . . . 54
20.4 Pseudoinverse constructions . . . 54
20.5 Condition estimation . . . 55
20.6 Block matrices . . . 55

21 Extended case studies **56**

21.1 Case 1: three-node AC network . . . 56
21.2 Case 2: four-node thermal ladder . . . 56
21.3 Case 3: absorbing Markov chain . . . 57
21.4 Case 4: weighted sensor fusion . . . 57
21.5 Case 5: constrained quadratic optimization . . . 57
21.6 Case 6: frame and celestial rotations . . . 58

VII Reference and troubleshooting **59**

22 DM42 matrix command atlas **60**

22.1 Creation and selection . . . 60
22.2 Indexed access . . . 60
22.3 Algebra and vector operations . . . 61
22.4 Operation semantics . . . 61
22.5 Capability boundary . . . 62

23 Troubleshooting, verification, and glossary **63**

23.1 A diagnostic decision path . . . 63
23.2 Fast symptom lookup . . . 63
23.3 Verification templates . . . 64
23.3.1 Matrix product . . . 64
23.3.2 Inverse . . . 64
23.3.3 Linear solve . . . 64
23.3.4 Rotation . . . 64
23.3.5 Markov matrix . . . 64
23.3.6 Least squares . . . 64
23.4 Independent numerical checks used in this edition . . . 65
23.5 Compact workflow cards . . . 65
23.6 Glossary . . . 65
23.7 Final checklist . . . 66

Bibliography **67**

List of Figures

2.1 Rows, columns, and the one-based entry a_{22} 5
2.2 The highlighted row and column give $1(7) + 2(9) + 3(11) = 58$ 6
2.3 A linear map scales oriented area by its determinant 7

3.1 The cross product is normal to the plane of a and b 9

9.1 A linear transformation maps the origin to itself and preserves lines 25

13.1 Two-dimensional system geometry 35

15.1 Least-squares line and the four measurements 40

16.1 A two-node resistive network 43
16.2 A chained three-dimensional rotation 45

18.1 Three-state transition network 48

23.1 Troubleshooting moves from representation to numerical evidence 63

List of Tables

1.1 What the DM42 does directly and what must be constructed 2

2.1 Matrix families worth recognizing at sight 5

4.1 Core matrix entry points 12

6.1 Indexed element and block commands 16

14.1 Common matrix failures and responses 38

19.1 Advanced factorization boundary 52

22.1 Creation, dimensions, and editor commands 60
22.2 Indexed commands 60
22.3 Algebraic commands 61
22.4 Binary operation semantics on matrices 61
22.5 Native, derived, and external methods 62

23.1 Symptoms, causes, and tests 63

Part I

Orientation and mathematical foundations

Chapter 1

Why matrices belong on the DM42

A matrix is a rectangular container for a linear relationship. That description is modest, but it covers electrical networks, rotations, chemical balances, least-squares fitting, Markov processes, and much more. The DM42 is unusually good at small and medium hand-worked problems because it combines a high-precision decimal core with named matrices, a matrix editor, complex arithmetic, and the RPN stack (SwissMicros GmbH, 2026b,c).

1.1 The calculator's numerical model

The DM42 build uses IEEE 754-2008 decimal128 arithmetic: 34 significant decimal digits and a very wide exponent range (IEEE, 2019; SwissMicros GmbH, 2026b). That is far more precision than a ten- or twelve-digit calculator, but it does not make an ill-conditioned problem well-conditioned. Precision reduces rounding error; it cannot restore information that the data never contained.

Three questions before every calculation

1. **Model:** Is the equation the right representation of the physical or statistical situation?
2. **Dimensions:** Are all products and sums conformable, and do units agree?
3. **Conditioning:** Would a small change in the data create a large change in the answer?

1.2 Native strengths and deliberate limits

Table 1.1: What the DM42 does directly and what must be constructed

Task	Native path	If more is required
Create, edit, resize	`NEWMAT`, `DIM`, `EDIT`, `EDITN`	Programmed data import/export
Solve square systems	SIMQ or matrix division	Residual and sensitivity checks
Determinant and inverse	`DET`, `INVRT`	Prefer a solve when only $A^{-1}b$ is needed
Transpose and norms	`TRANS`, `FRNM`, `RNRM`	Build conjugate transpose or other norms manually
Vector products	`DOT`, `CROSS`, `UVEC`	Enforce the intended complex inner product yourself
Eigenvalues, QR, SVD	No general native command	Small analytic construction or external software

The absence of a native command is not a defect: it is a boundary. General eigenvalue solvers, QR factorization, Cholesky factorization, singular-value decomposition, pseudoinverses, matrix exponentials, and reduced row-echelon form are not exposed as built-in matrix commands in the current DM42 command table.

Do not infer a command from a textbook name

If this handbook says "construct" or "program," the result is assembled from native operations; it is not a hidden DM42 function. The quick reference in chapter 22 separates native commands from derived methods.

1.3 RPN orientation

Binary operations use the stack order $Y:X$: Y is the left operand and X the right operand. Therefore:

$$Y - X, \qquad YX, \qquad Y/X.$$

For matrices, multiplication is the usual product. Matrix division has a special, useful meaning:

$$\boxed{Y/X = X^{-1}Y}$$

implemented by solving $XR = Y$ for R. This is not YX^{-1}.

One stack snapshot

To compute AB, recall A, then recall B, then multiply.

Step	Y	X
RCL A	—	A
RCL B	A	B
×	—	AB

Reversing the recalls computes BA, which is generally different.

1.4 Command entry and program entry

In ordinary mode, a named command may be selected from a menu or executed with XEQ ▸ COMMAND. In program mode, XEQ is an instruction-entry key: it places `XEQ "name"` in the program. It never means "run this now" while editing a program.

DM42 display : program listing

```
00  { 12-Byte Prgm }
01  LBL "AXPY"
02  RCL "A"
03  RCL "X"
04  ×
05  RCL+ "B"
06  RTN
```

How to read sequences in this book

2 ▸ ENTER ▸ 3 ▸ NEW means press the shown keys in that order. A printed function such as XEQ ▸ FRNM means execute it in normal mode. In a program listing, `XEQ "FRNM"` is a stored instruction.

Checkpoint: first-session setup

1. Confirm whether the calculator is in normal or PRGM mode.
2. Decide whether the four-level or big stack is active.
3. Locate MATRIX, `DIM?`, and the catalog search.
4. Create a 2×2 matrix, store it as `A`, recall it, and inspect its dimensions.
5. Delete the temporary variable only after the recall succeeds.

Chapter 2

Matrix foundations

2.1 Shape, entries, and indexing

An $m \times n$ matrix has m rows and n columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

The entry a_{ij} lies in row i, column j. The DM42 uses this one-based convention in its editor display and in `STOIJ`/`RCLIJ`. Internally, the implementation stores values in a row-major stream, but the user interface remains one-based.

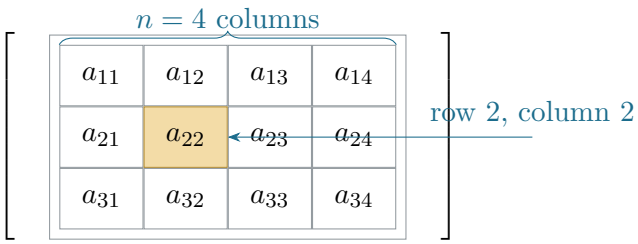


Figure 2.1: Rows, columns, and the one-based entry a_{22}

2.2 Important matrix families

Table 2.1: Matrix families worth recognizing at sight

Family	Condition	Why it matters
Row / column vector	$1 \times n$ / $m \times 1$	Direction, data, coefficients, right-hand sides
Square	$m = n$	Determinants, inverses, eigenvalues
Diagonal	$a_{ij} = 0$ for $i \neq j$	Scaling and decoupled systems
Identity	$I_{ii} = 1$, others 0	Multiplicative neutral element
Triangular	Zero above or below diagonal	Efficient substitution and determinant
Symmetric	$A^{\mathsf{T}} = A$	Energy, covariance, undirected networks

Family	Condition	Why it matters
Hermitian	$A^{\mathsf{H}} = A$	Complex analogue of symmetry
Orthogonal	$Q^{\mathsf{T}}Q = I$	Length-preserving real transformations
Unitary	$Q^{\mathsf{H}}Q = I$	Length-preserving complex transformations
Singular	$\det A = 0$	No unique inverse or square-system solution
Sparse	Mostly zeros	Model structure; the DM42 still stores a dense matrix

2.3 Equality, addition, and scalar multiplication

Matrices are equal only when their shapes and corresponding entries agree. Addition and subtraction require equal shapes:

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

Scalar multiplication changes every entry, not the shape:

$$(\alpha A)_{ij} = \alpha a_{ij}.$$

Dimensions are a proof tool

Write shapes beside symbols. If A is $m \times n$ and B is $n \times p$, then AB is $m \times p$. This simple annotation catches most setup errors before the calculator does.

2.4 Matrix multiplication

The i, j entry of AB is a row–column dot product:

$$(AB)_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

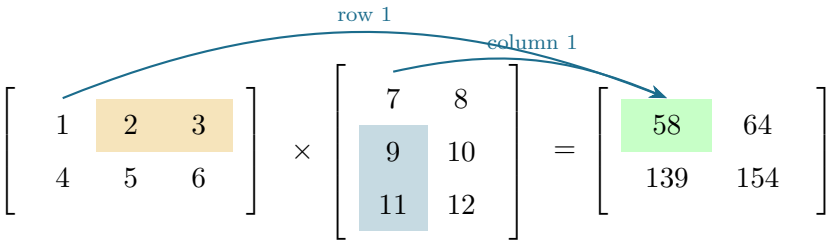


Figure 2.2: The highlighted row and column give $1(7) + 2(9) + 3(11) = 58$

Multiplication is associative and distributive, but usually not commutative:

$$(AB)C = A(BC), \qquad A(B + C) = AB + AC, \qquad AB \neq BA.$$

2.5 Transpose and conjugate transpose

The transpose exchanges rows and columns: $(A^{\mathsf{T}})_{ij} = a_{ji}$. For complex matrices, the conjugate transpose is

$$A^{\mathsf{H}} = \overline{A^{\mathsf{T}}}.$$

The DM42's native `TRANS` performs the first operation only. On a complex matrix it moves entries without conjugating them.

Complex inner-product convention

The native `DOT` on two complex matrices computes the bilinear sum $\sum_i a_i b_i$, with no conjugation. If the intended inner product is $\langle a, b\rangle = a^{\mathsf{H}} b = \sum_i \overline{a_i} b_i$, explicitly conjugate the first vector before `DOT`. This distinction can change both phase and magnitude.

2.6 Determinant, inverse, rank, and linear independence

For a 2×2 matrix,

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc, \qquad A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

when $ad - bc \neq 0$. Geometrically, $|\det A|$ is the area or volume scale factor. A zero determinant means a square matrix collapses at least one dimension.

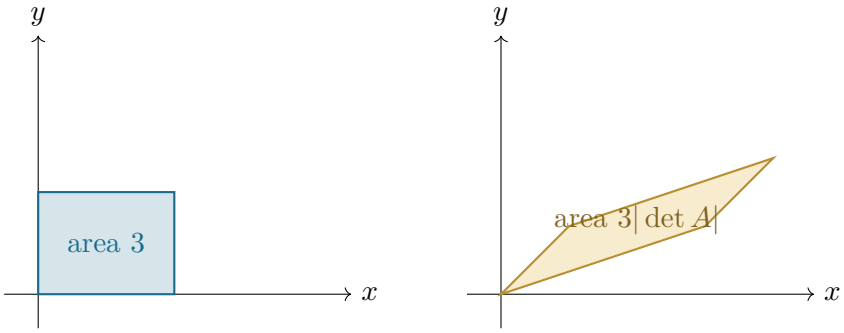


Figure 2.3: A linear map scales oriented area by its determinant

Rank is the number of independent directions carried by a matrix. The current DM42 has no general native `RANK` command. For small matrices, rank can be inferred from determinants of minors or from elimination; for serious rank decisions, use a scale-aware tolerance and an SVD-capable system (Golub and Loan, 2013; Higham, 2002).

Chapter 3

Vectors: geometry carried in matrices

3.1 Column vectors and the matrix view

A vector $v = (v_1, \ldots, v_n)^\mathsf{T}$ is naturally an $n \times 1$ matrix. On the DM42, a row and column vector are both matrices, and native `DOT` compares their flattened row-major element streams as long as the total element count matches. Shape still matters for multiplication, so a 1×3 row and 3×1 column behave differently in products.

3.2 Length, direction, and projection

The Euclidean length is

$$\|v\|_2 = \sqrt{v^\mathsf{T} v}.$$

For real matrix vectors, `FRNM` returns this same value, and `UVEC` returns $v/\|v\|_2$. A zero vector has no direction, so `UVEC` reports `Invalid Data`.

The projection of b onto a nonzero a is

$$\operatorname{proj}_a(b) = \frac{a^\mathsf{T} b}{a^\mathsf{T} a} a.$$

Normalize and project

Let

$$a = \begin{bmatrix} 3 \\ 4 \\ 0 \end{bmatrix}, \qquad b = \begin{bmatrix} 2 \\ -1 \\ 2 \end{bmatrix}.$$

Then $\|a\| = 5$, $\widehat{a} = (0.6, 0.8, 0)^\mathsf{T}$, $a^\mathsf{T} b = 2$, and

$$\operatorname{proj}_a(b) = \frac{2}{25} a = \begin{bmatrix} 0.24 \\ 0.32 \\ 0 \end{bmatrix}.$$

1. Recall a, then execute **XEQ** ▸ `FRNM`; expect 5.
2. Recall a, then execute **XEQ** ▸ `UVEC`; expect a 3×1 matrix.
3. Recall a, recall b, then execute **XEQ** ▸ `DOT`; expect 2.

3.3 The cross product

For three-component real vectors,

$$a \times b = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}.$$

The result is perpendicular to both inputs and follows the right-hand rule.

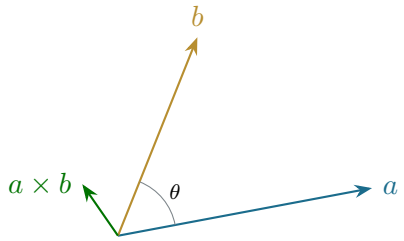


Figure 3.1: The cross product is normal to the plane of a and b

For the vectors above,

$$a \times b = \begin{bmatrix} 8 \\ -6 \\ -11 \end{bmatrix}.$$

The native `CROSS` accepts real matrices containing at most three total elements, reads them in row-major order, pads short inputs with zeros, and returns a 1×3 real row matrix. It does not accept complex matrices. This is broader than "only 3×1 vectors" but narrower than a general complex vector product.

Checkpoint: orthogonality

Compute $(a \times b) \cdot a$ and $(a \times b) \cdot b$. Both should be zero apart from rounding. Also check $\|a \times b\| = \|a\|\|b\| |\sin\theta|$.

3.4 Torque and work

If a force F acts at position r, the torque is $\tau = r \times F$. Work is $W = F \cdot d$. The same two native vector products therefore encode fundamentally different geometry.

A mechanics check

Let $r = (0.30, 0.10, 0)$ m and $F = (0, 40, 10)$ N. Then

$$\tau = r \times F = (1, -3, 12)\ \text{N m}, \qquad \|\tau\| = \sqrt{154} \approx 12.4097\ \text{N m}.$$

Enter both vectors as real matrices and use `CROSS`. Confirm the returned row vector is 1×3, even if the inputs were columns.

3.5 Chapter review

Checkpoint: foundations

1. Why can AB exist while BA does not?
2. What difference between `TRANS` and A^{H} matters for complex work?
3. What result does $Y : X$ matrix division compute?
4. Which DM42 command normalizes a nonzero real vector?

Answers: inner dimensions; conjugation; $X^{-1}Y$; `UVEC`.

Part II

The DM42 matrix workspace

Chapter 4

Creating and naming matrices

4.1 The Matrix menu

The official quick reference groups matrix work under the MATRIX application. The visible soft-key rows vary with context, so this handbook gives both the menu intention and the command name. If a soft key is not visible, use catalog search or XEQ ▸ COMMAND in normal mode.

Table 4.1: Core matrix entry points

Command	Purpose	Stack or argument
NEWMAT	Create a zero-filled real matrix on the stack	Y: rows, X: columns
DIM	Create or resize a named matrix	Y: rows, X: columns, then name
DIM?	Recall dimensions	returns rows to Y, columns to X
EDIT	Edit the matrix in X	matrix remains associated with stack editor
EDITN	Edit a named matrix	name argument
INDEX	Select a named matrix for indexed commands	name argument

4.2 Create a matrix on the stack

To create a 2×3 real zero matrix:

2 ▸ ENTER ▸ 3 ▸ MATRIX ▸ NEW.

The underlying command is NEWMAT. The result in X is

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Store it under a short, descriptive name:

STO ▸ A.

DM42 display : after creating 2×3

```
Y:
X:  [ 2 × 3 Matrix ]
```

Rows first, columns second

The first number is the row count and the second is the column count. A common mistake is to think in screen width first and create the transpose of the intended matrix.

4.3 Create or resize a named matrix

The named workflow avoids a temporary stack matrix:

2 ▸ ENTER ▸ 3 ▸ MATRIX ▸ ▼ ▸ DIM ▸ ENTER ▸ A ▸ ENTER.

Equivalently, execute `DIM` with the name argument. If `A` does not exist or is not already a matrix/list, a real zero matrix is created. If it is a real or complex matrix, it is resized.

4.3.1 What resize preserves

`DIM` preserves the existing row-major element stream. Suppose

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Resizing from 2×3 to 3×2 does not preserve a visual rectangle; the total size is unchanged, so the same stream $1, 2, 3, 4, 5, 6$ is reinterpreted:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}.$$

Growing appends zeros to the stream; shrinking discards the tail. Plan a backup before destructive reshaping.

Checkpoint: resize prediction

Predict the result of resizing the original 2×3 matrix to 2×4. Answer:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 0 & 0 \end{bmatrix},$$

because the storage stream is preserved, not the old row boundaries.

4.4 Inspect shape

Recall a matrix and execute XEQ ▸ `DIM?`. The command returns two real values:

$$Y = \text{rows}, \qquad X = \text{columns}.$$

Use `DIM?` as a preflight before multiplication or before a program accepts user data.

Chapter 5

The matrix editor

5.1 Entering and leaving edit mode

Use `EDIT` to edit the matrix in X, or `EDITN` to edit a named matrix. The editor starts at $I = 1, J = 1$. The current element appears in X, while the editor keeps the target matrix and the I, J pointers in its state.

DM42 display : representative editor state

A[2,3]
Y:
X: 6.0000
I=2 J=3 WRAP

Enter an element and advance with ENTER. In the default `WRAP` mode, moving right from the last column continues at the first column of the next row; leaving the last element returns to the first. `GROW` mode adds a row when you advance beyond the end. The editor automatically selects wrap when it is entered or exited (SwissMicros GmbH, 2026c; Okken, 2026).

A reliable data-entry rhythm

Prepare the values in row-major order. Start `EDITN`, type each value, and press ENTER. After the last value, stop and verify several corner entries; do not press through the boundary unless wrap behavior is intentional.

5.2 Editing a 3×3 example

Create and fill

$$K = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 3 \end{bmatrix}.$$

1. Create named K as 3×3 with `DIM`.
2. Execute `EDITN K`.
3. Enter $4, -1, 0, -1, 4, -1, 0, -1, 3$ in that order.
4. Exit the editor and recall K. Use `DIM?`; expect 3 in Y and 3 in X.
5. Execute `TRANS` and compare with K; a symmetric matrix should be unchanged.

5.3 Complex matrices

A new matrix is real. Convert the matrix in X to complex by adding complex zero using the documented sequence:

0 ▸ ENTER ▸ COMPLEX ▸ +.

For a named matrix, recall it, perform the conversion, and store it back. Once complex, entries accept real or complex scalars. A real value stored into a complex element receives zero imaginary part.

No mixed element type inside a complex matrix

A matrix is real or complex as a whole. Adding complex zero converts the complete real matrix. `TRANS` does not conjugate the elements; build A^{H} by elementwise conjugation followed by `TRANS`.

5.4 Strings in real matrices

The underlying HP-42S-compatible data model can store strings in real matrices. Many numerical commands then report `Alpha Data Is Invalid`. For numerical work, treat a matrix as homogeneous numeric data and keep labels in variable names or documentation.

Checkpoint: editor integrity

After a long entry session, inspect the four corners, one interior entry, the matrix type, and `DIM?`. For complex data, include at least one entry with a nonzero imaginary part so an accidental real-matrix workflow is obvious.

Chapter 6

Indexed access and structural editing

6.1 Select an indexed matrix

Execute `INDEX` with a named matrix. The calculator remembers the target and initializes its pointers. Indexed commands then act on that matrix without recalling the whole matrix to X.

XEQ ▸ INDEX ▸ A ▸ ENTER

6.2 Set and recall I, J

To move to row i, column j, put i in Y and j in X, then execute `STOIJ`:

2 ▸ ENTER ▸ 3 ▸ XEQ ▸ STOIJ.

`RCLIJ` returns the current row in Y and current column in X. Indices are one-based; zero, negative, or out-of-range indices produce a dimension error.

Table 6.1: Indexed element and block commands

Command	Action	Stack behavior
`RCLEL`	Recall current element	pushes scalar
`STOEL`	Store X in current element	target type rules apply
`GETM`	Extract a block starting at I, J	Y: row count, X: column count
`PUTM`	Insert matrix X at I, J	destination must contain the block
`RCLIJ`	Recall pointers	row to Y, column to X
`STOIJ`	Set pointers	row in Y, column in X

6.3 Element operations

After indexing A, recall a_{23}:

2 ▸ ENTER ▸ 3 ▸ XEQ ▸ STOIJ ▸ XEQ ▸ RCLEL.

To replace it with -7.5, set the pointers, put -7.5 in X, and execute `STOEL`.

Checkpoint: pointer discipline

Always treat the pointer state as part of the calculation. Before a destructive `STOEL`, `PUTM`, `INSR`, or `DELR`, use `RCLIJ` and verify the returned pair.

6.4 Extract and insert submatrices

Let

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}.$$

Index A, set $I = 2, J = 2$, place 2 in Y and 3 in X, then execute `GETM`. The result is

$$\begin{bmatrix} 6 & 7 & 8 \\ 10 & 11 & 12 \end{bmatrix}.$$

The requested rectangle must fit completely inside the source matrix.

For `PUTM`, put the source block in X after setting the destination's I, J. A real block can be inserted into a complex destination and receives zero imaginary parts. A complex block cannot be inserted into a real destination.

6.5 Insert, delete, and swap rows

`INSR` inserts a zero row at the current indexed row and shifts that row and all following rows downward. `DELR` deletes the current row and shifts later rows upward. A one-row matrix cannot lose its only row.

`R<>R` swaps two rows of the indexed matrix. Put one row number in Y and the other in X; because swapping is symmetric, their order does not change the outcome.

Manual elimination with row operations

For

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 3 & 4 & 11 \end{array}\right],$$

the conceptual operation $R_2 \leftarrow R_2 - 3R_1$ gives

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 0 & -2 & -4 \end{array}\right].$$

The DM42 has row swap but no single native "add a multiple of another row" matrix command. For a small pedagogical elimination, recall/store each element through indexed access; for repeated work, write a loop. For solving, native matrix division is more direct and usually more stable.

6.6 Safe structural workflow

1. Duplicate the matrix under a temporary name.
2. `INDEX` the temporary matrix.
3. `RCLIJ` and verify the target row.
4. Apply `INSR`, `DELR`, or `R<>R`.
5. Check `DIM?` and inspect boundary elements.
6. Only then replace the original.

Post-edit integrity audit

After a row insertion or deletion, use `DIM?`, recall the first and last entries of the affected rows, and confirm that the indexed pointer remains where the next operation expects. After `R<>R`, compare one distinctive element from each swapped row.

Chapter 7

Storage, stack, and program patterns

7.1 Named matrices versus stack matrices

A stack matrix is convenient for a short calculation; a named matrix is safer for a model. Use short names that encode roles:

`A` coefficients, `B` right-hand sides, `X` solution, `R` residual.

Commands generally create new matrix results. Storing intermediate values protects against accidental stack loss and makes dimension checks explicit.

7.2 The four-level and big stack

The DM42 can use the classic four-level RPN stack or a larger stack configuration. Matrix commands obey the selected stack model. In the classic stack, plan temporary values carefully: a binary matrix operation consumes X and Y and returns one result. In a large stack, history is easier to preserve, but names remain the most legible form of state.

7.3 A reusable matrix program

The affine operation $y = Ax + b$ appears everywhere. Store A, x, b as named matrices and use:

Listing 7.1: Affine matrix-vector evaluation

```
LBL "AFFINE"
RCL "A"
RCL "X"
×
RCL "B"
+
STO "Y"
RTN
```

To enter the program, switch to PRGM. On the line that calls a named subroutine, pressing XEQ inserts an `XEQ` instruction. It does not run the subroutine during entry.

Program-entry example with XEQ

Suppose the helper program RESID computes $b - Ax$. In program mode, the intended line is:

```
XEQ "RESID"
```

Press XEQ, select the alpha name RESID, and terminate the name. The program listing should show the instruction. Exit program mode before testing it.

7.4 Dimension guard pattern

The current language has DIM?, but comparing and branching on both returned dimensions requires care with stack order. A robust high-level pattern is:

Listing 7.2: Pseudocode for a multiplication guard

```
RCL "A"
XEQ "DIMS" ; stores AR and AC
RCL "B"
XEQ "DIMS" ; stores BR and BC
RCL "AC"
RCL "BR"
X=Y?
GTO 01
"DIM ERROR"
AVIEW
STOP
LBL 01
RCL "A"
RCL "B"
×
RTN
```

This listing is a pattern, not a claim that DIMS is native: it is a user subroutine built around DIM?.

7.5 Programmatic element traversal

Index the target matrix, set $I = 1, J = 1$, and traverse with editor movement or explicit pointer arithmetic. For reliability:

1. store row and column counts before starting;
2. set WRAP or GROW intentionally;
3. do not rely on a stale indexed target left by another program;
4. restore or document the pointer state on return.

The indexed matrix is global calculator state

INDEX changes which matrix subsequent indexed commands address. A program that assumes the caller's indexed target is fragile. Select its own target at entry and avoid destructive commands unless the target is explicit.

7.6 Checkpoint project: build and inspect

Create

$$M = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \\ 0 & 0 & -1 \end{bmatrix}.$$

Then:

1. verify M is 4×3;
2. extract rows 2–4 and columns 2–3 with `GETM`;
3. transpose the block;
4. compute its Frobenius norm;
5. store the result as `B`;
6. confirm B is 2×3.

The extracted block is

$$\begin{bmatrix} 2 & -1 \\ -1 & 2 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \end{bmatrix}, \quad \|B\|_F = \sqrt{11} \approx 3.3166247903554.$$

Part III

Core operations and geometric meaning

Chapter 8

Arithmetic on the RPN stack

8.1 Addition and subtraction

Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 5 & -1 \\ 2 & 0 \end{bmatrix}.$$

Equal dimensions permit elementwise addition and subtraction:

$$A + B = \begin{bmatrix} 6 & 1 \\ 5 & 4 \end{bmatrix}, \qquad A - B = \begin{bmatrix} -4 & 3 \\ 1 & 4 \end{bmatrix}.$$

1. RCL A, then RCL B.
2. Press + for $A + B$, or - for $A - B$.
3. Store the result before starting another branch.

Adding or subtracting a scalar and a matrix maps the scalar across all entries. This can be useful, but it is not a standard textbook matrix-plus-scalar operation. Document it when it appears in a model.

8.2 Scalar multiplication and division

For scalar $\alpha = 2.5$,

$$\alpha A = \begin{bmatrix} 2.5 & 5 \\ 7.5 & 10 \end{bmatrix}.$$

Put the matrix in Y and the scalar in X, then press ×. Scalar/matrix and matrix/scalar combinations are elementwise. Matrix/matrix division is different and solves a system; never infer its behavior from scalar division.

8.3 Matrix multiplication

For the same A, B:

$$AB = \begin{bmatrix} 9 & -1 \\ 23 & -3 \end{bmatrix}, \qquad BA = \begin{bmatrix} 2 & 6 \\ 2 & 4 \end{bmatrix}.$$

Checkpoint: stack order

Compute and store AB, then compute BA. If the results match by accident in another example, that does not prove commutativity; try a second pair or reason from the factors.

8.4 Powers

For a square matrix, repeated multiplication creates $A^2, A^3, \ldots$. The calculator's scalar power key is not a general matrix-power interface; multiply matrices explicitly or write exponentiation by squaring.

Exponentiation by squaring

For integer $n \geq 0$, keep an accumulator $R = I$. While $n > 0$: if n is odd, set $R \leftarrow RA$; set $A \leftarrow A^2$; halve n and discard the remainder. This uses $O(\log n)$ matrix multiplications.

A power with an exact check

For the Fibonacci matrix

$$F = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \qquad F^{10} = \begin{bmatrix} 89 & 55 \\ 55 & 34 \end{bmatrix}.$$

Exponentiation by squaring needs F^2, F^4, F^8, followed by $F^8 F^2$. The symmetry and adjacent Fibonacci numbers provide quick entry checks.

Chapter 9

Linear transformations

9.1 A matrix as a machine

The product $y = Ax$ maps an input vector to an output vector. Columns of A are the images of the coordinate basis vectors. This column view makes transformation matrices easier to build and debug.

9.1.1 Two-dimensional primitives

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \quad S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}, \quad H_x = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}.$$

Reflections are obtained by negating one coordinate or reflecting across a line through the origin.

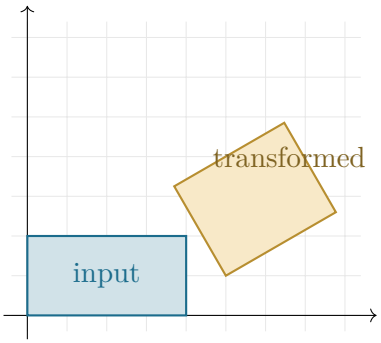


Figure 9.1: A linear transformation maps the origin to itself and preserves lines

9.2 Composition order

If x is first scaled by S and then rotated by R, the combined matrix is RS:

$$x \longmapsto Sx \longmapsto R(Sx) = (RS)x.$$

The rightmost factor acts first.

Scale and rotate

Use $x = (1, 2)^{\mathsf{T}}$, $S = \operatorname{diag}(2, 0.5)$, and $\theta = 30°$. Then

$$Sx = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad R(30°)Sx = \begin{bmatrix} 1.23205080756888 \\ 1.86602540378444 \end{bmatrix}.$$

Reversing the order gives

$$SR(30^\circ)x = \begin{bmatrix} -0.267949192431123 \\ 1.11602540378444 \end{bmatrix}.$$

Store R, S, x; compute both branches and compare.

9.3 Homogeneous coordinates

Translation is not linear in ordinary 2-vectors, but it becomes a matrix operation in homogeneous coordinates:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

This unifies rotation, scale, shear, reflection, and translation into one 3×3 product.

Affine workflow

Build each elementary 3×3 matrix, multiply them in reverse action order, and apply the composite once. Preserve the final coordinate 1; if it drifts from 1, the matrix or vector was entered incorrectly.

Chapter 10

Determinants, inverses, and division

10.1 Determinants

Recall a square matrix and execute XEQ ▸ DET. A non-square matrix gives Dimension Error. A matrix containing string data gives Alpha Data Is Invalid.

A determinant check

$$A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}, \qquad \det A = 4(6) - 7(2) = 10.$$

On the DM42: RCL A ▸ XEQ ▸ DET. Compare the native result with the hand formula before trusting a longer workflow.

10.2 Inverse

For the same matrix,

$$A^{-1} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}.$$

Use XEQ ▸ INVRT. Verify by multiplication:

$$AA^{-1} \approx I, \qquad A^{-1}A \approx I.$$

The residual from identity is a better test than visual inspection.

Singular-matrix setting

DM42 configuration includes a Singular Matrix Error option. With the option enabled, inversion or solving an exactly singular system reports Singular Matrix. With HP-compatible behavior enabled instead, an attempted singular inverse can contain huge placeholder values. A small or zero determinant and a large residual remain essential checks.

10.3 Why solving beats forming an inverse

To compute $x = A^{-1}b$, use matrix division with b in Y and A in X:

RCL B ▸ RCL A ▸ ÷.

The implementation solves $Ax = b$ directly. Explicitly calculating A^{-1} adds work and usually adds rounding error (Golub and Loan, 2013; Higham, 2002).

10.4 Multiple right-hand sides

If $B = [b_1\ b_2\ \cdots\ b_p]$, then

$$X = A^{-1}B$$

solves all p systems at once. Put B in Y and A in X, divide, and obtain a matrix whose columns are the individual solutions.

Two right-hand sides

With

$$A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

matrix division B/A returns A^{-1}. This special case explains the division convention: it solves $AX = B$, not $XA = B$.

Chapter 11

Norms, sums, and complex matrices

11.1 Frobenius and row norms

For

$$A = \begin{bmatrix} 1 & -2 \\ 3 & 4 \end{bmatrix},$$

the Frobenius norm and maximum absolute row-sum norm are

$$\|A\|_F = \sqrt{1^2 + (-2)^2 + 3^2 + 4^2} = \sqrt{30} \approx 5.477225575,$$

$$\|A\|_\infty = \max(1 + 2, 3 + 4) = 7.$$

The DM42 commands are `FRNM` and `RNRM`, respectively. `FRNM` supports real and complex matrices. For a complex matrix it sums squared magnitudes. `RNRM` sums element magnitudes across each row and returns the maximum.

11.2 Row sums

`RSUM` maps an $m \times n$ matrix to an $m \times 1$ column of algebraic row sums:

$$\texttt{RSUM}(A) = \begin{bmatrix} -1 \\ 7 \end{bmatrix}.$$

For complex matrices, real and imaginary parts are summed separately.

11.3 Complex representation

An AC network, phasor state, or quantum amplitude often needs complex matrices. Write

$$A = A_R + iA_I.$$

The DM42 stores complex entries directly and supports complex addition, multiplication, division/solve, determinant, inverse, transpose, row sums, and Frobenius/row norms.

11.3.1 Transpose is not adjoint

For

$$A = \begin{bmatrix} 1 + i & 2 \\ 3i & 4 - i \end{bmatrix},$$

native TRANS returns

$$A^{\mathsf{T}} = \begin{bmatrix} 1+i & 3i \\ 2 & 4-i \end{bmatrix},$$

whereas the conjugate transpose is

$$A^{\mathsf{H}} = \begin{bmatrix} 1-i & -3i \\ 2 & 4+i \end{bmatrix}.$$

11.3.2 Complex dot product

Let $u = (1+i,\, 2)$ and $v = (i,\, 3-i)$. Native DOT returns

$$u^{\mathsf{T}}v = (1+i)i + 2(3-i) = 5 - i.$$

The Hermitian inner product instead gives

$$u^{\mathsf{H}}v = (1-i)i + 2(3-i) = 7 - i.$$

Conjugate u explicitly if the latter is required.

11.4 Conjugation construction

For a complex scalar $z = x + iy$, $\overline{z} = x - iy$. For a matrix, traverse elements, conjugate each scalar, and store it into a duplicate. Then use TRANS if an adjoint is needed.

Listing 11.1: Conceptual user routine for a conjugate transpose

```
; Input: named complex matrix A
; Output: AH = conjugate-transpose(A)
RCL "A"
STO "AH"
INDEX "AH"
; loop over I=1..rows, J=1..columns:
RCLEL
CONJ ; user scalar helper or x-iy construction
STOEL
; end loop
RCL "AH"
XEQ "TRANS" ; entered as an XEQ instruction in PRGM mode
STO "AH"
RTN
```

No native general matrix conjugation command

CONJ in the listing is a user helper, not a claimed DM42 matrix command. For one-off work, conjugate entries during data entry. For repeated work, implement and test a scalar conjugation loop.

11.5 Norm-based checks

Given a computed inverse B, form $E = I - AB$ and calculate $\|E\|_F$. Given a computed solution x, form $r = b - Ax$ and calculate $\|r\|_F$. A small residual confirms the calculation fits the entered data; it does not prove the data or model are correct.

Checkpoint: core operations

1. What does `RNRM` measure?
2. Why is B/A preferable to first forming A^{-1} when solving $AX = B$?
3. Does `TRANS` conjugate complex entries?
4. What matrix does I/A return?

Answers: maximum absolute row sum; direct solve; no; A^{-1}.

Part IV

Linear systems and numerical reliability

Chapter 12

Solving square systems

12.1 The model $Ax = b$

A system of n linear equations in n unknowns is

$$\underbrace{\begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}}_{b}.$$

The DM42 offers two natural routes: the guided SIMQ application and matrix division.

12.2 The SIMQ application

Select MATRIX, then SIMQ, and choose the equation count n. SIMQ creates or resizes three named matrices:

$$\texttt{MATA} : n \times n, \qquad \texttt{MATB} : n \times 1, \qquad \texttt{MATX} : n \times 1.$$

Use the MATA and MATB entries to edit coefficients and right-hand side. Select MATX to solve and inspect the result.

SIMQ is not an XEQ command

SIMQ is a hidden, non-programmable, non-assignable application command reached from its menu. Do not write `XEQ "SIMQ"` in a program. For programmable solving, use named matrices and matrix division.

If `MATA`, `MATB`, or `MATX` already exists as a real or complex matrix, SIMQ duplicates and resizes it, preserving the row-major stream as described earlier. Back up valuable matrices before changing the equation count.

12.3 Worked 2×2 system

Solve

$$\begin{aligned} 2x + y &= 5, \\ x - y &= 1. \end{aligned} \qquad A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 1 \end{bmatrix}.$$

1. Open MATRIX → SIMQ; choose 2.

2. Edit `MATA`: enter $2, 1, 1, -1$.
3. Edit `MATB`: enter $5, 1$.
4. Select `MATX`. The solution is $(2, 1)^{\mathsf{T}}$.
5. Verify $Ax - b = (0, 0)^{\mathsf{T}}$.

DM42 display : solution matrix

MATX[1,1]
X: 2.00000000000000
next element: 1.00000000000000

12.4 The programmable solve

Store A and b, then compute b/A:

RCL B ▸ RCL A ▸ ÷ ▸ STO X.

Listing 12.1: Square solve and residual

```
LBL "SOLVE"
RCL "B"
RCL "A"
÷
STO "X"
RCL "A"
RCL "X"
×
RCL "B"
-
STO "R"
RTN
```

The subtraction in this listing forms $Ax - b$; $b - Ax$ is equally valid for a norm check.

12.5 Complex systems

If either `MATA` or `MATB` is complex, the solution is complex. Convert a real matrix by adding complex zero before entry. The coefficient matrix must remain square and its row count must match the right-hand side.

Chapter 13

Unique, inconsistent, and underdetermined systems

13.1 Geometric classification

For two equations in two unknowns, each equation is a line:

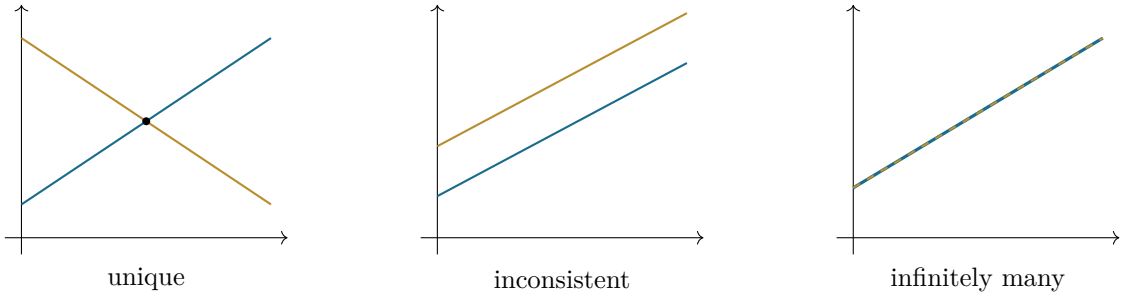


Figure 13.1: Two-dimensional system geometry

For a square system:

$$\det A \neq 0 \iff \operatorname{rank} A = n \iff \text{one solution.}$$

When A is singular, compare $\operatorname{rank} A$ and $\operatorname{rank}[A \mid b]$:

$$\operatorname{rank} A < \operatorname{rank}[A \mid b] \Rightarrow \text{no solution,}$$

$$\operatorname{rank} A = \operatorname{rank}[A \mid b] < n \Rightarrow \text{infinitely many solutions.}$$

13.2 Two diagnostic examples

13.2.1 Inconsistent

$$\begin{aligned} x + y &= 2, \\ 2x + 2y &= 5. \end{aligned}$$

The second left side duplicates the first at twice the scale, but the right side should then be 4, not 5. No vector satisfies both.

13.2.2 Infinitely many

$$x + y = 2,$$
$$2x + 2y = 4.$$

The second equation adds no new information. Let $y = t$; then $x = 2 - t$.

What the DM42 tells you

`DET` returns zero for an exactly singular square matrix. A solve can report `Singular Matrix` when that setting is enabled. These indicate failure of uniqueness, but they do not by themselves distinguish inconsistency from infinitely many solutions. Inspect the augmented equations or perform elimination.

13.3 Rectangular systems

For $m > n$, there may be more equations than unknowns; least squares finds the closest fit. For $m < n$, there are fewer equations than unknowns; a solution, if one exists, is not unique without an additional criterion. Native matrix division requires a square divisor, so use the constructions in chapter 15 or external software for larger or delicate rectangular problems.

Chapter 14

Conditioning, precision, and diagnostics

14.1 Residual versus forward error

Suppose $\widehat{x}$ is the computed solution. The residual is

$$r = b - A\widehat{x}.$$

A backward-stable algorithm can produce a very small residual even when $\widehat{x}$ is far from the exact solution of the intended data. The condition number links input perturbations to possible solution changes:

$$\frac{\|\delta x\|}{\|x\|} \lesssim \kappa(A)\left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|}\right).$$

14.2 A transparent ill-conditioned example

Let

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-12} \end{bmatrix}, \qquad b = \begin{bmatrix} 2 \\ 2 + 10^{-12} \end{bmatrix}.$$

The exact decimal solution is $x = (1, 1)^{\mathsf{T}}$. Now perturb only the second entry of b by 10^{-15}:

$$\widetilde{b} = \begin{bmatrix} 2 \\ 2 + 10^{-12} + 10^{-15} \end{bmatrix}.$$

Subtracting the equations gives $10^{-12}x_2 = 10^{-12} + 10^{-15}$, so

$$\widetilde{x} = \begin{bmatrix} 0.999 \\ 1.001 \end{bmatrix}.$$

A relative data change near 5×10^{-16} created a relative solution change near 10^{-3}. The infinity-norm condition number is approximately 4×10^{12}.

Why 34 digits do not end the discussion

Decimal128 can represent the displayed decimal data very accurately, but the matrix still magnifies uncertainty already present in measurement or rounding. More arithmetic precision protects the calculation; it does not improve the experimental data.

14.3 A practical diagnostic ladder

1. **Scale check:** compare magnitudes in each row and column.
2. **Determinant check:** useful for small square matrices, but do not use the raw determinant alone across different scales.
3. **Residual check:** compute $r = b - A\widehat{x}$, then $\|r\|_F$.
4. **Relative residual:**
$$\eta = \frac{\|r\|}{\|A\|\|\widehat{x}\| + \|b\|}.$$
5. **Perturbation check:** change uncertain inputs within plausible tolerances and resolve.
6. **Cross-method check:** for a small case, compare with hand elimination or independent software.

14.4 Scaling

If one equation contains coefficients near 10^9 and another near 10^{-6}, rescale units or equations before solving. Row scaling does not change the exact solution if the right-hand side is scaled with the row. Column scaling changes the unknown variables and must be undone afterward.

14.5 Error messages as evidence

Table 14.1: Common matrix failures and responses

Message / symptom	Likely cause	Corrective action
`Dimension Error`	Incompatible product, bad index, nonsquare determinant/inverse, block overflow	Check `DIM?`, I, J, and inner dimensions
`Invalid Type`	Scalar where matrix required, complex block into real target, unsupported command type	Inspect X/Y types; convert destination intentionally
`Alpha Data Is Invalid`	String encountered in numeric operation	Replace string elements with numbers
`Singular Matrix`	Coefficient matrix lacks a unique inverse	Classify system; do not suppress the diagnosis
`Insufficient Memory`	Matrix or temporary allocation too large	Reduce dimensions, remove temporaries, split the task
`Huge inverse entries`	Singular-error compatibility mode or severe ill-conditioning	Enable singular error; inspect determinant and residual

Chapter 15

Least squares and fitting

15.1 The normal-equation construction

For an overdetermined real system $Ax \approx b$, a least-squares solution minimizes

$$\|Ax - b\|_2^2.$$

If A has full column rank, the normal equations are

$$(A^{\mathsf{T}}A)x = A^{\mathsf{T}}b.$$

On the DM42:

1. Store A and b.
2. Compute and store A^{T} with `TRANS`.
3. Form $N = A^{\mathsf{T}}A$.
4. Form $c = A^{\mathsf{T}}b$.
5. Solve $Nx = c$ by putting c in Y and N in X, then dividing.
6. Compute $r = b - Ax$ and verify $A^{\mathsf{T}}r \approx 0$.

Normal equations square the condition number

$\kappa(A^{\mathsf{T}}A) \approx \kappa(A)^2$. The construction is reasonable for small, well-scaled, full-rank problems; it is not a substitute for QR or SVD on difficult data (Golub and Loan, 2013; Higham, 2002). The DM42 has no native general QR/SVD solver.

15.2 Line fit example

Fit $y = \beta_0 + \beta_1 x$ to

$$(x, y) = (0, 1.1), (1, 2.9), (2, 5.2), (3, 6.8).$$

Use

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 1.1 \\ 2.9 \\ 5.2 \\ 6.8 \end{bmatrix}.$$

Then

$$A^{\mathsf{T}}A = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix}, \qquad A^{\mathsf{T}}b = \begin{bmatrix} 16 \\ 33.7 \end{bmatrix},$$

and

$$\widehat{\beta} = \begin{bmatrix} 1.09 \\ 1.94 \end{bmatrix}.$$

The residual is

$$r = \begin{bmatrix} 0.01 \\ -0.13 \\ 0.23 \\ -0.11 \end{bmatrix}, \quad \|r\|_2 \approx 0.2863564213, \quad A^{\mathsf{T}}r \approx 0.$$

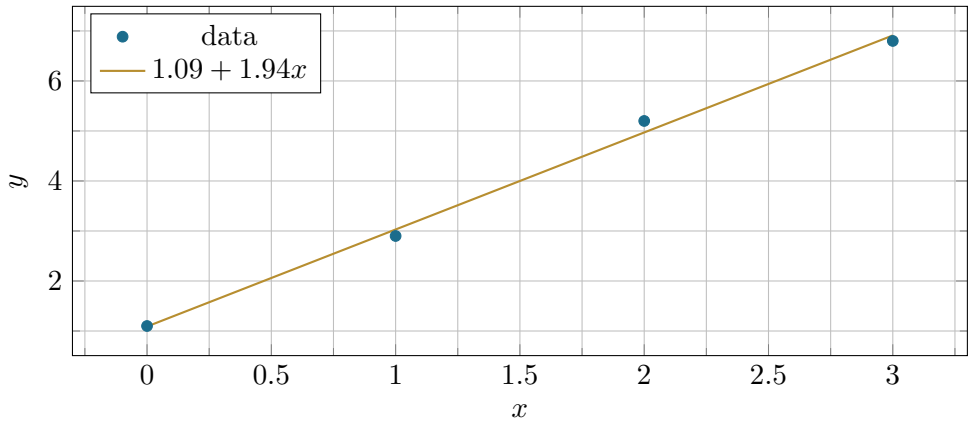


Figure 15.1: Least-squares line and the four measurements

15.3 Centering and scaling a fit

For large x-values, define $u = (x - x_0)/s$, fit in u, then convert coefficients back. Centering reduces correlation between intercept and slope columns and often improves the normal equations.

15.4 Underdetermined minimum norm

For $A \in \mathbb{R}^{m \times n}$ with $m < n$ and full row rank, the minimum-norm solution is

$$x = A^{\mathsf{T}}(AA^{\mathsf{T}})^{-1}b.$$

This can be assembled on the DM42 for small, well-conditioned cases. It is a derived formula, not a native pseudoinverse command, and it inherits the numerical weaknesses of normal-equation-like products.

Checkpoint: systems and fitting

1. Which three matrices does SIMQ manage?
2. Can SIMQ be entered as `XEQ "SIMQ"` in a program?
3. What does a small residual fail to prove?
4. What diagnostic identity should a least-squares residual satisfy?

Answers: `MATA`, `MATB`, `MATX`; no; that the model/data are accurate or the forward error is small; $A^{\mathsf{T}}r \approx 0$.

When to move beyond the handheld

Escalate the calculation when rank is uncertain, the normal equations are visibly ill-conditioned, the matrix is large enough that entry verification dominates the work, or the result controls safety-critical action. Preserve the DM42 model and residual as a compact independent check.

Part V

Applied models

Chapter 16

Networks in engineering and physics

16.1 DC nodal analysis

Kirchhoff's current law makes nodal analysis a matrix problem. In the network below, conductances contribute positively to diagonal entries and negatively between connected nodes (Hayt et al., 2019).

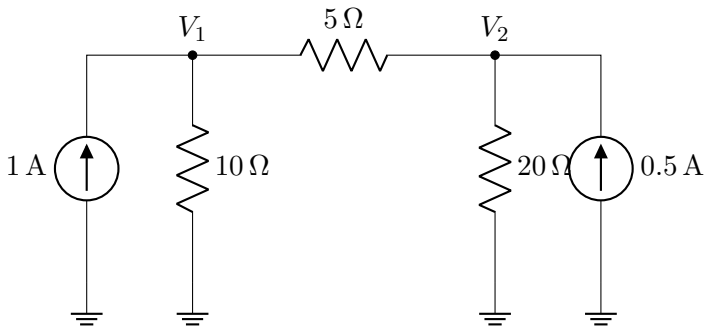


Figure 16.1: A two-node resistive network

The conductance matrix and injection vector are

$$G = \begin{bmatrix} 1/10 + 1/5 & -1/5 \\ -1/5 & 1/20 + 1/5 \end{bmatrix} = \begin{bmatrix} .30 & -.20 \\ -.20 & .25 \end{bmatrix}, \qquad i = \begin{bmatrix} 1 \\ .5 \end{bmatrix}.$$

Solving $Gv = i$ gives

$$v = \begin{bmatrix} 10 \\ 10 \end{bmatrix} \text{ V}.$$

Checkpoint: physical verification

The bridge current is $(V_1 - V_2)/5 = 0$. Node 1 sends 1 A through its 10 Ω resistor; node 2 sends 0.5 A through its 20 Ω resistor. KCL is satisfied at both nodes.

16.2 AC phasor analysis

Replace conductance by complex admittance. Consider

$$Y = \begin{bmatrix} 0.045 + 0.010i & -0.025 - 0.010i \\ -0.025 - 0.010i & 0.035 - 0.010i \end{bmatrix} \text{ S}, \qquad I = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ A}.$$

After converting both matrices to complex type, solve $YV = I$:

$$V \approx \begin{bmatrix} 27.4888558692 + 5.6463595840i \\ 13.5215453195 + 15.7503714710i \end{bmatrix} \text{ V}.$$

1. Enter Y as a 2×2 complex matrix and I as 2×1.
2. Put I in Y and Y in X; press ÷.
3. Store V, compute $YV - I$, and apply `FRNM`.
4. Convert each voltage to magnitude/phase only after the rectangular solve.

16.3 Thermal resistance network

At steady state, thermal conductances create the same matrix structure:

$$\begin{bmatrix} 12 & -2 \\ -2 & 7 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \end{bmatrix} = \begin{bmatrix} 220 \\ 95 \end{bmatrix}.$$

The solution is $T_1 = 21.625$, $T_2 = 19.75$ in the chosen temperature unit. Negative off-diagonal terms represent heat exchange between nodes. A positive diagonal larger than the sum of couplings is a useful physical sanity check.

16.4 Statics and force balance

Force equilibrium in two dimensions is

$$\sum_{k} F_{x,k} = 0, \qquad \sum_{k} F_{y,k} = 0.$$

Unknown member forces become columns of a coefficient matrix whose entries are direction cosines. Moment balance adds another row. Before solving, use `UVEC` on each direction vector and verify its norm is one.

16.5 Rigid-body rotation

Three-dimensional rotation matrices are orthogonal with determinant $+1$. With a Z–Y–X active rotation,

$$R = R_z(30^\circ) R_y(20^\circ) R_x(-10^\circ)$$

is approximately

$$\begin{bmatrix} .81379768 & -.54383814 & .20487413 \\ .46984631 & .82317294 & .31879578 \\ -.34202014 & -.16317591 & .92541658 \end{bmatrix}.$$

Applied to $v = (1, 2, 3)^{\mathsf{T}}$,

$$Rv \approx (0.34074378,\ 3.07257953,\ 2.10787777)^{\mathsf{T}}.$$

Check $\det R \approx 1$, $R^{\mathsf{T}} R \approx I$, and $\|Rv\| = \|v\|$.

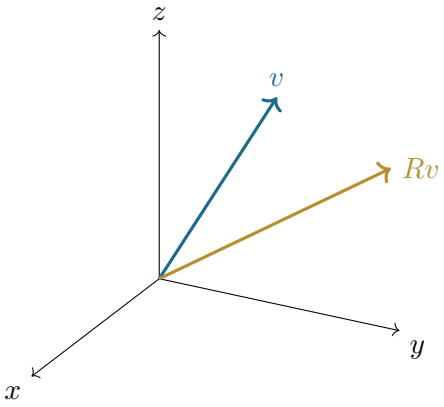


Figure 16.2: A chained three-dimensional rotation

Chapter 17

Chemistry, economics, and decisions

17.1 Balancing a chemical equation

Balance propane combustion:

$$a\,\mathrm{C_3H_8} + b\,\mathrm{O_2} \longrightarrow c\,\mathrm{CO_2} + d\,\mathrm{H_2O}.$$

Element conservation gives

$$\begin{bmatrix} 3 & 0 & -1 & 0 \\ 8 & 0 & 0 & -2 \\ 0 & 2 & -2 & -1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = 0.$$

This homogeneous system is underdetermined. Fix the scale with $a = 1$, move its column to the right, and solve the remaining 3×3 system. The result is

$$(a, b, c, d) = (1, 5, 3, 4).$$

The general null-space problem has no native DM42 command; fixing one coefficient turns this small example into a square solve.

17.2 Portfolio mean and variance

For weights w, expected returns μ, and covariance matrix Σ,

$$\mu_p = w^{\mathsf{T}}\mu, \qquad \sigma_p^2 = w^{\mathsf{T}}\Sigma w.$$

Use

$$w = \begin{bmatrix} .40 \\ .35 \\ .25 \end{bmatrix}, \quad \mu = \begin{bmatrix} .06 \\ .09 \\ .04 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} .040 & .012 & .006 \\ .012 & .090 & .009 \\ .006 & .009 & .025 \end{bmatrix}.$$

Then

$$\mu_p = 0.0655, \qquad \sigma_p^2 = 0.0251225, \qquad \sigma_p \approx 0.1585007886.$$

Compute $s = \Sigma w$, then $w^{\mathsf{T}}s$. Verify Σ is symmetric and the weights sum to one.

Not investment advice

This is a linear-algebra example. Estimated returns and covariances are uncertain, can change over time, and may not describe tail risk or transaction constraints.

17.3 A two-strategy zero-sum game

Let the row player's payoff matrix be

$$P = \begin{bmatrix} 3 & 0 \\ 1 & 2 \end{bmatrix}.$$

If the row player chooses row 1 with probability p, make the column player indifferent:

$$3p + 1(1-p) = 0p + 2(1-p),$$

giving $p = 0.25$. If the column player chooses column 1 with probability q, make the row player indifferent:

$$3q = 1q + 2(1-q),$$

giving $q = 0.5$. The game value is 1.5. These are tiny linear systems plus probability constraints.

17.4 Leontief input–output model

If C is the technical coefficient matrix and d final demand, gross output satisfies

$$x = Cx + d, \qquad (I - C)x = d.$$

Build $I - C$ explicitly and solve. A plausible model requires nonnegative C, d, x, and the production system must be productive; in small cases, check that the solve is stable and the answer is nonnegative.

17.5 Sensitivity from repeated solves

To measure how an output $x = A^{-1}b$ changes with a parameter, perturb that parameter by $\pm h$ and use

$$\frac{\partial x}{\partial p} \approx \frac{x(p+h) - x(p-h)}{2h}.$$

Choose h large enough to dominate rounding yet small relative to the model scale. Reusing the same named matrices makes a DM42 sensitivity sweep transparent.

Chapter 18

Markov chains, coordinates, and data fusion

18.1 Markov transition matrices

For a row-state vector p_k and row-stochastic matrix P,

$$p_{k+1} = p_k P, \qquad p_n = p_0 P^n.$$

Let

$$P = \begin{bmatrix} .7 & .2 & .1 \\ .1 & .8 & .1 \\ .2 & .3 & .5 \end{bmatrix}, \qquad p_0 = (1, 0, 0).$$

Repeated multiplication gives

$$p_5 = (.34112,\ .49392,\ .16496),$$
$$p_{20} \approx (.29168952,\ .54164382,\ .16666666).$$

Every iterate should have nonnegative entries summing to one.

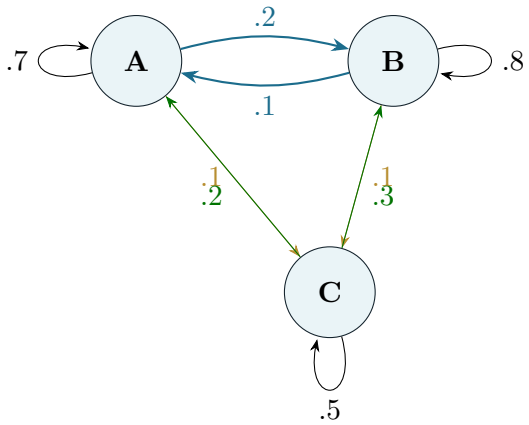


Figure 18.1: Three-state transition network

18.2 Stationary distribution

A stationary row distribution satisfies $\pi P = \pi$ and $\sum_i \pi_i = 1$. Solve

$$(P^{\mathsf{T}} - I)\pi^{\mathsf{T}} = 0$$

after replacing one redundant row with $(1, 1, 1)$ and the corresponding right-hand side with 1. Here

$$\pi = \left(\frac{7}{24}, \frac{13}{24}, \frac{1}{6}\right) \approx (.29166667, .54166667, .16666667).$$

Compare with a long repeated-power result (Grinstead and Snell, 1997).

18.3 Astronomical coordinate rotation

A standard ICRS/J2000 Cartesian unit vector can be rotated into Galactic Cartesian coordinates with

$$R_{\mathrm{G}\leftarrow\mathrm{I}} = \begin{bmatrix} -.0548755604 & -.8734370902 & -.4838350155 \\ .4941094279 & -.4448296300 & .7469822445 \\ -.8676661490 & -.1980763734 & .4559837762 \end{bmatrix}.$$

For the north celestial pole $v_I = (0, 0, 1)^{\mathsf{T}}$,

$$v_G = (-.4838350155,\ .7469822445,\ .4559837762)^{\mathsf{T}}.$$

Converting back to longitude and latitude yields approximately

$$\ell = 122.93192^\circ, \qquad b = 27.12825^\circ.$$

Conventions matter: confirm active/passive direction, epoch/frame, angle units, and row/column orientation against the coordinate system in use (Astropy Collaboration, 2026).

18.4 Sensor fusion by weighted least squares

For measurements $z = Hx + \varepsilon$ with positive-definite weight matrix W, the weighted least-squares estimate is

$$\widehat{x} = (H^{\mathsf{T}} W H)^{-1} H^{\mathsf{T}} W z.$$

On the DM42, form

$$N = H^{\mathsf{T}} W H, \qquad c = H^{\mathsf{T}} W z,$$

then solve $N\widehat{x} = c$. Use diagonal $W = \mathrm{diag}(1/\sigma_i^2)$ when independent measurement variances are known. Avoid explicit inversion; solve.

Covariance propagation needs the right transpose

For real linear propagation, $P_y = A P_x A^{\mathsf{T}}$. For complex random vectors, many conventions require A^{H}, which the native `TRANS` does not provide. Build the adjoint explicitly.

18.5 Logistics and flow balance

An incidence matrix B maps edge flows f to node imbalances:

$$Bf = s.$$

Each edge column contains one $+1$ and one -1 under a chosen orientation. One balance equation is redundant in a connected closed network, so remove a row or impose a reference/optimization criterion before solving.

Checkpoint: applications

1. Why do network matrices often have negative off-diagonal entries?
2. What two invariants should a Markov iterate satisfy?
3. What checks certify a three-dimensional rotation matrix?
4. Why is a chemical balance naturally homogeneous?

Answers: coupling between nodes; nonnegative entries and unit sum; $R^{\mathsf{T}}R \approx I$ and $\det R \approx 1$; atom conservation has zero net production per element.

18.6 Application handoff checklist

Before reporting an applied result, record the matrix convention, variable order, units, source-data precision, and residual or invariant. For coordinates, add frame and epoch; for circuits, frequency and phasor convention; for stochastic models, row- versus column-stochastic orientation; for fitted data, the weight definition and residual norm.

Part VI

Advanced constructions

Chapter 19

Factorizations and eigenproblems

19.1 LU factorization: present internally, not exposed generally

Gaussian elimination factors a matrix conceptually as

$$PA = LU,$$

where P records row permutations, L is lower triangular, and U is upper triangular. The DM42 implementation uses LU-like machinery for determinant, inverse, and solve, but it does not expose a general user command returning P, L, U. Use the native solve rather than manually reconstructing factors unless the factors themselves are the subject.

19.2 Cholesky, QR, and SVD boundaries

Table 19.1: Advanced factorization boundary

Method	Best use	DM42 route
Cholesky	Symmetric positive-definite systems	User implementation for small matrices; no native factor command
QR	Stable least squares, orthogonal bases	User construction is possible but laborious
SVD	Rank, pseudoinverse, difficult least squares	Use external software for general cases
LU with pivoting	General square systems	Used behind native solve; factors not returned

19.3 Analytic 2×2 eigenvalues

For

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

the characteristic polynomial is

$$\lambda^2 - (a+d)\lambda + (ad - bc) = 0.$$

Thus

$$\lambda_{1,2} = \frac{\operatorname{tr} A \pm \sqrt{(\operatorname{tr} A)^2 - 4 \det A}}{2}.$$

For

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

$\operatorname{tr} A = 4$, $\det A = 3$, so $\lambda_1 = 3$, $\lambda_2 = 1$. Solve $(A - \lambda I)v = 0$ by choosing a free component:

$$v_1 \propto (1, 1)^{\mathsf{T}}, \qquad v_2 \propto (1, -1)^{\mathsf{T}}.$$

Use `UVEC` to normalize real eigenvectors.

No general native eigensolver

`DET` and ordinary arithmetic support the 2×2 formula, but there is no general built-in `EIG` command. Repeated roots, nearly defective matrices, and higher degrees need more careful algorithms than the characteristic polynomial.

19.4 Power iteration

The dominant eigenvector can be approximated by

$$v_{k+1} = \frac{Av_k}{\|Av_k\|}.$$

For

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}, \qquad v_0 = (1, 0)^{\mathsf{T}},$$

normalizing by the largest component gives

$$v_1 = (1, .5)^{\mathsf{T}}, \quad v_3 \approx (1, .90698)^{\mathsf{T}}, \quad v_6 \approx (1, .99387)^{\mathsf{T}}.$$

The scale factors approach the dominant eigenvalue 5, and the vector approaches $(1, 1)^{\mathsf{T}}$.

Listing 19.1: Conceptual power-iteration loop

```
; A and V are named real matrices
LBL "POWER"
RCL "A"
RCL "V"
×
STO "W"
XEQ "FRNM" ; XEQ inserts this command in PRGM mode
STO "N"
RCL "W"
RCL "N"
÷
STO "V"
RTN
```

The listing shows one iteration. A surrounding loop and a convergence test on successive vectors or Rayleigh quotients are required.

19.5 Rayleigh quotient

For nonzero real v,

$$\rho(v) = \frac{v^{\mathsf{T}} A v}{v^{\mathsf{T}} v}.$$

Near an eigenvector, $\rho(v)$ estimates the associated eigenvalue. Compute the numerator with matrix products or `DOT`; compute the denominator with `DOT` or `FRNM`^{2}.

Chapter 20

Iterative and derived matrix methods

20.1 Stationary iteration

Split $A = D + L + U$. Jacobi iteration is

$$x^{(k+1)} = D^{-1}\big(b - (L + U)x^{(k)}\big).$$

Gauss–Seidel uses newly updated components as soon as they are available. On a handheld, these are most useful for teaching, exploiting a special sparse structure, or checking a direct solve. Convergence is not automatic; strict diagonal dominance is a common sufficient condition.

20.2 Matrix exponential

The general matrix exponential

$$e^{A} = \sum_{k=0}^{\infty} \frac{A^{k}}{k!}$$

has no native command. For a diagonal matrix, exponentiate diagonal entries. For a diagonalizable $A = V\Lambda V^{-1}$,

$$e^{A} = V e^{\Lambda} V^{-1},$$

but only if the eigendecomposition is reliable. For a small matrix, a scaled Taylor series can be programmed with a remainder test; for serious work, use a scaling-and-squaring implementation with Padé approximants.

20.3 Cayley–Hamilton shortcuts

Every 2×2 matrix satisfies

$$A^{2} - (\operatorname{tr} A)A + (\det A)I = 0.$$

Therefore every higher power reduces to $\alpha A + \beta I$. This is practical on the DM42: calculate $\operatorname{tr} A$ manually from diagonal entries, use `DET`, and recurse on two scalar coefficients instead of multiplying full matrices repeatedly.

20.4 Pseudoinverse constructions

For full column rank A,

$$A^{+} = (A^{\mathsf{T}}A)^{-1}A^{\mathsf{T}}.$$

For full row rank,

$$A^{+} = A^{\mathsf{T}}(AA^{\mathsf{T}})^{-1}.$$

These formulas are educational and workable for small, well-scaled matrices. They should not be used to determine numerical rank; a robust pseudoinverse uses the SVD and a documented tolerance.

20.5 Condition estimation

The exact condition number in a chosen norm is

$$\kappa(A) = \|A\|\|A^{-1}\|.$$

For a small nonsingular matrix, estimate $\kappa_{\infty}(A)$ with two native `RNRM` calls, one on A and one on `INVRT`(A). This requires the inverse and is therefore more expensive than a professional condition estimator, but it is transparent.

A scale-sensitive condition estimate

For

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-12} \end{bmatrix},$$

$\|A\|_{\infty} = 2 + 10^{-12}$, and $\|A^{-1}\|_{\infty}$ is about 2×10^{12}. Their product is about 4×10^{12}, matching the sensitivity observed in the perturbation example.

20.6 Block matrices

Partitioning can reveal structure:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} f \\ g \end{bmatrix}.$$

If A is invertible, eliminate x and solve with the Schur complement

$$S = D - CA^{-1}B.$$

On the DM42, avoid forming A^{-1}: solve $AZ = B$ and $Au = f$, then use $S = D - CZ$ and $g - Cu$.

Chapter 21

Extended case studies

21.1 Case 1: three-node AC network

Let branch admittances be

$$y_{12} = .020 + .010i, \qquad y_{23} = .015 - .005i,$$

and shunt admittances

$$y_{1g} = .010, \quad y_{2g} = .005 + .020i, \quad y_{3g} = .020 - .010i.$$

The nodal matrix is

$$Y = \begin{bmatrix} .030 + .010i & -.020 - .010i & 0 \\ -.020 - .010i & .040 + .025i & -.015 + .005i \\ 0 & -.015 + .005i & .035 - .015i \end{bmatrix}.$$

For $I = (1,\ .5 - .2i,\ 0)^{\mathsf{T}}$ A, matrix division gives

$$V \approx \begin{bmatrix} 55.09992946 - 28.93957207i \\ 31.35198683 - 31.53538679i \\ 14.06066306 - 11.96802257i \end{bmatrix} \text{ V}.$$

Verify $YV - I$ with `FRNM`; an independent binary-double calculation gives a residual below 5×10^{-16}.

21.2 Case 2: four-node thermal ladder

Solve

$$\begin{bmatrix} 8 & -3 & 0 & 0 \\ -3 & 9 & -2 & 0 \\ 0 & -2 & 7 & -1 \\ 0 & 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 100 \\ 40 \\ 25 \\ 10 \end{bmatrix}.$$

The result is

$$T \approx \begin{bmatrix} 16.91036236 \\ 11.76096631 \\ 7.55880483 \\ 4.38970121 \end{bmatrix}.$$

The monotone decrease is physically plausible for this forcing and boundary arrangement. Check each row as a heat balance.

21.3 Case 3: absorbing Markov chain

For three transient states and one absorbing state, take

$$Q = \begin{bmatrix} .5 & .3 & .1 \\ .2 & .5 & .2 \\ .1 & .3 & .4 \end{bmatrix}, \qquad R = \begin{bmatrix} .1 \\ .1 \\ .2 \end{bmatrix}.$$

The fundamental matrix is the derived construction

$$N = (I - Q)^{-1} \approx \begin{bmatrix} 3.58209 & 3.13433 & 1.64179 \\ 2.08955 & 4.32836 & 1.79104 \\ 1.64179 & 2.68657 & 2.83582 \end{bmatrix}.$$

Expected steps to absorption are

$$N\mathbf{1} \approx (8.35821,\ 8.20896,\ 7.16418)^{\mathsf{T}}.$$

Because there is only one absorbing state, $NR = \mathbf{1}$, a useful exact check.

21.4 Case 4: weighted sensor fusion

Estimate two state components from

$$H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad z = \begin{bmatrix} 2.1 \\ -.9 \\ 1.05 \\ 2.8 \end{bmatrix},$$

with standard deviations $(.1, .2, .15, .1)$. Let $W = \operatorname{diag}(1/\sigma_i^2)$. Form

$$N = H^{\mathsf{T}} W H = \begin{bmatrix} 244.444444 & -55.555556 \\ -55.555556 & 169.444444 \end{bmatrix},$$

$$c = H^{\mathsf{T}} W z = (536.666667,\ -255.833333)^{\mathsf{T}}.$$

Solving $N\widehat{x} = c$ gives

$$\widehat{x} \approx (2.00144928,\ -.85362319)^{\mathsf{T}}.$$

Check the weighted orthogonality condition $H^{\mathsf{T}} W r \approx 0$.

21.5 Case 5: constrained quadratic optimization

Minimize

$$\frac{1}{2} x^{\mathsf{T}} \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} x - \begin{bmatrix} 2 & 1 \end{bmatrix} x \quad \text{subject to} \quad x_1 + x_2 = 1.$$

The first-order KKT system is

$$\begin{bmatrix} 2 & 0 & 1 \\ 0 & 4 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}.$$

The solution is

$$x_1 = \frac{5}{6}, \qquad x_2 = \frac{1}{6}, \qquad \lambda = \frac{1}{3}.$$

The constraint residual is exactly zero. KKT matrices can be indefinite; a zero determinant or unstable result requires reformulation or a more capable solver (Nocedal and Wright, 2006).

21.6 Case 6: frame and celestial rotations

Combine the Z–Y–X frame rotation from chapter 16 only after verifying each elementary rotation. For a celestial transform, verify the ICRS-to-Galactic matrix by checking

$$RR^{\mathsf{T}} \approx I, \qquad \det R \approx 1, \qquad \|Rv\| \approx \|v\|.$$

Then convert Cartesian coordinates to angles with the correct quadrants:

$$\ell = \operatorname{atan2}(y, x), \qquad b = \arcsin(z)$$

for a unit vector. A transposed rotation or swapped atan2 arguments can produce a plausible but wrong sky position.

Part VII

Reference and troubleshooting

Chapter 22

DM42 matrix command atlas

22.1 Creation and selection

Table 22.1: Creation, dimensions, and editor commands

Command	Input	Result	Notes
NEWMAT	Y rows; X columns	real zero matrix in X	Stack object; store it if needed
DIM	Y rows; X columns; name	creates/resizes named matrix	Preserves row-major prefix; fills growth with zero
DIM?	matrix in X	Y rows; X columns	Matrix remains consumed as unary input
EDIT	matrix in X	interactive editor	Defaults to wrap; target is stack matrix
EDITN	name	interactive editor	Edits named object
INDEX	name	selects indexed target	Required by indexed commands
WRAP	—	wrap traversal mode	Does not grow matrix
GROW	—	grow traversal mode	Adds a row at end during entry

22.2 Indexed access

Table 22.2: Indexed commands

Command	Input	Result	Notes
STOIJ	Y row; X column	sets I, J	One-based; must be in range
RCLIJ	—	Y row; X column	Requires active indexed target
RCLEL	—	pushes current element	Strings possible only in real matrix model
STOEL	scalar/string in X	stores current element	Complex scalar requires complex target
GETM	Y rows; X columns	submatrix in X	Top-left is current I, J

Command	Input	Result	Notes
PUTM	source matrix in X	updates target	Block must fit; complex-to-real rejected
INSR	—	inserts zero row	At current indexed row
DELR	—	deletes current row	Cannot delete sole matrix row
R<>R	Y row; X row	swaps rows	Works for real and complex matrices

22.3 Algebra and vector operations

Table 22.3: Algebraic commands

Command	Domain	Output	Critical detail
DET	square real/complex matrix	scalar	Exact singular determinant is zero
INVRT	square real/complex matrix	inverse matrix	Behavior on singularity depends on setting
TRANS	real/complex matrix	transposed matrix	Never conjugates complex entries
FRNM	real/complex matrix	scalar	Frobenius norm; complex magnitudes included
RNRM	real/complex matrix	scalar	maximum absolute row sum
RSUM	real/complex matrix	$m \times 1$ matrix	algebraic row sums
DOT	two equal-size flattened matrices	real/complex scalar	complex case is bilinear, no conjugation
CROSS	two real matrices, each ≤ 3 elements	1×3 row	zero-pads short vectors
UVEC	nonzero real vector/matrix	normalized matrix	uses Frobenius norm
SIMQ	menu plus equation count	manages MATA/B/X	hidden, non-programmable application

22.4 Operation semantics

Table 22.4: Binary operation semantics on matrices

Operands	Operation	Meaning
Matrix, matrix	$+$, $-$	elementwise; equal shape required
Matrix, matrix	$\times$	ordinary matrix product
Y matrix, X matrix	$\div$	solve $XR = Y$, returning $R = X^{-1}Y$
Matrix, scalar	arithmetic	scalar mapped across elements
Scalar, matrix	arithmetic	scalar mapped with operand order preserved

22.5 Capability boundary

Table 22.5: Native, derived, and external methods

Task	Native	Derived	Guidance
Square solve	✓	—	matrix division or SIMQ
Least squares	×	✓	normal equations only for small, well-conditioned data
General eigenproblem	×	limited	analytic 2×2, power iteration
LU factors	×	possible	internal solve uses factorization; factors not returned
QR / Cholesky	×	possible	user programs; verify carefully
SVD / robust rank	×	×	external numerical system
Pseudoinverse	×	limited	full-rank formulas; SVD preferred
Matrix exponential	×	limited	special matrices or programmed approximation
Conjugate transpose	×	✓	conjugate elements, then `TRANS`

Chapter 23

Troubleshooting, verification, and glossary

23.1 A diagnostic decision path

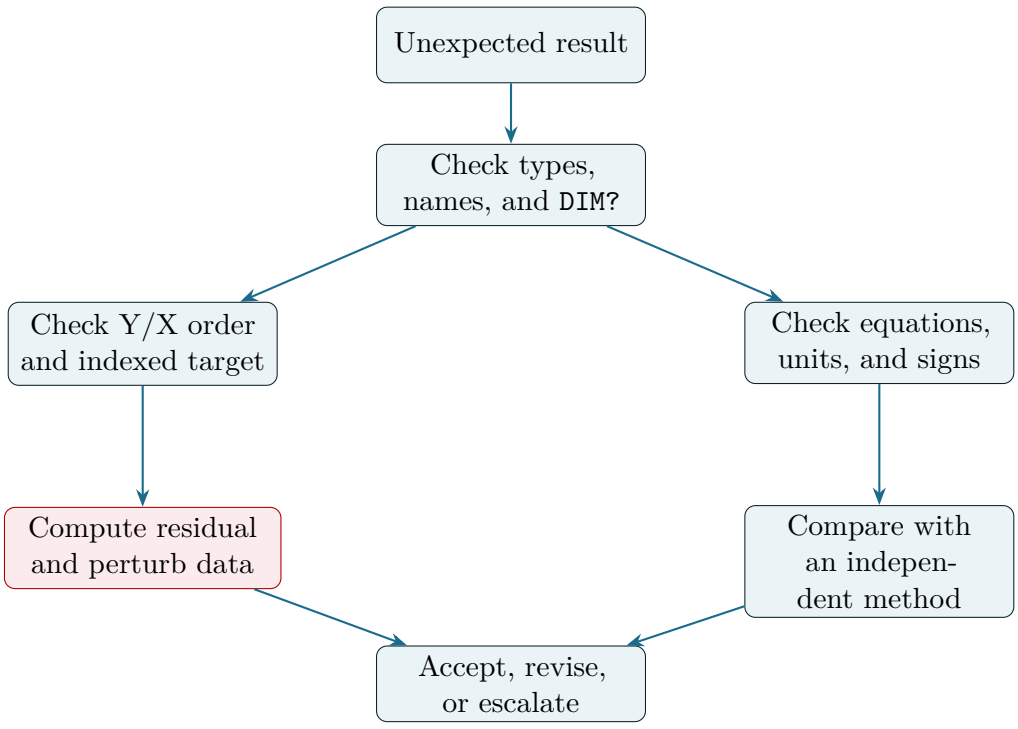


Figure 23.1: Troubleshooting moves from representation to numerical evidence

23.2 Fast symptom lookup

Table 23.1: Symptoms, causes, and tests

Symptom	Probable cause	Immediate test
Product has wrong shape	Factors recalled in reverse order	Write dimensions above both factors

Symptom	Probable cause	Immediate test
Plausible but wrong solve	Matrix division order reversed	Confirm divisor X is coefficient matrix
Complex projection wrong	Native `DOT` lacked conjugation	Compute one term by hand
Data moved after `DIM`	Row-major reshaping	Flatten old matrix and regroup
`PUTM` fails	Block exceeds boundary or type mismatch	Check I, J, block size, destination type
`UVEC` fails	Zero input or unsupported type	Check `FRNM`; inspect matrix type
Inverse contains huge values	Singularity or severe conditioning	Enable singular error; check residual
SIMQ unavailable by `XEQ`	It is a menu-only hidden command	Use matrix division in a program
Program ran while being entered	Left PRGM mode or misunderstood key state	Inspect stored listing before execution

23.3 Verification templates

23.3.1 Matrix product

$$C = AB, \qquad c_{ij} \stackrel{?}{=} a_{i,:} b_{:,j}.$$

Check at least one corner and one interior entry by hand.

23.3.2 Inverse

$$E_L = I - AA^{-1}, \qquad E_R = I - A^{-1}A.$$

Both should be small. Checking only one side can miss an operand-order mistake.

23.3.3 Linear solve

$$r = b - A\widehat{x}, \qquad \eta = \frac{\|r\|}{\|A\|\|\widehat{x}\| + \|b\|}.$$

23.3.4 Rotation

$$R^{\mathsf{T}}R \approx I, \qquad \det R \approx 1, \qquad \|Rv\| \approx \|v\|.$$

23.3.5 Markov matrix

$$P_{ij} \geq 0, \qquad \sum_j P_{ij} = 1$$

for a row-stochastic convention; state vectors should retain unit sum.

23.3.6 Least squares

$$A^{\mathsf{T}}r \approx 0.$$

This certifies the normal-equation stationarity condition, not model validity.

23.4 Independent numerical checks used in this edition

All printed numerical examples were recomputed independently with array linear algebra before typesetting. Exact small examples were also checked symbolically or by substitution. The verification suite covered:

- every displayed square-system solution and residual;
- the real and complex network examples;
- least-squares coefficients and orthogonality;
- Markov iterates, stationary distribution, and absorption times;
- rotation orthogonality, determinant, and transformed vectors;
- portfolio quadratic form and constrained KKT solve.

23.5 Compact workflow cards

Solve $Ax = b$

Store b in `B` and A in `A`. Then

RCL B ▸ RCL A ▸ ÷ ▸ STO X.

Verify $AX - B$ with `FRNM`.

Extract a block

`INDEX` the named source. Put top-left row in Y and column in X; `STOIJ`. Put requested row count in Y and column count in X; `GETM`.

Least-squares line or model

Form A^{T}, $N = A^{\mathsf{T}}A$, $c = A^{\mathsf{T}}b$. Solve $Nx = c$. Verify $A^{\mathsf{T}}(b - Ax) \approx 0$. Escalate to QR/SVD software if rank or conditioning is uncertain.

Complex adjoint

Duplicate the matrix, conjugate every entry with indexed access, then apply `TRANS`. Native `TRANS` alone is only a transpose.

23.6 Glossary

Adjoint / conjugate transpose
$A^{\mathsf{H}} = \overline{A^{\mathsf{T}}}$. Must be constructed for DM42 complex matrices.

Condition number
Worst-case sensitivity multiplier for relative input perturbations.

Determinant
Signed area/volume scale factor of a square linear map.

Frobenius norm
Square root of the sum of squared entry magnitudes.

Hermitian
Complex matrix satisfying $A^{\mathsf{H}} = A$.

Ill-conditioned
Highly sensitive to small input changes.

Indexed matrix
Named matrix selected by `INDEX` for element/block commands.

Inner product
A scalar product; in complex spaces it normally conjugates the first argument.

Null space
Vectors x satisfying $Ax = 0$.

Orthogonal matrix
Real square Q with $Q^{\mathsf{T}}Q = I$.

Rank
Number of independent rows or columns.

Residual
Difference between observed right side and the one predicted by a computed solution.

Row-major
Storage order that lists all entries of row 1, then row 2, and so on.

Singular
Square matrix with no inverse.

Unitary matrix
Complex square U with $U^{\mathsf{H}}U = I$.

23.7 Final checklist

1. Did I write the model and units before entering values?
2. Are dimensions conformable?
3. Is Y/X order correct?
4. Is the indexed target explicit?
5. If complex, do I need conjugation?
6. Did a resize reinterpret row boundaries?
7. Is a direct solve preferable to an inverse?
8. What residual or invariant will I check?
9. Could conditioning dominate the result?
10. Have I preserved enough significant digits and documented the source data?

The durable habit

The strongest DM42 workflow is not a long key sequence. It is a short loop: model, dimension, compute, verify, interpret. The calculator supplies precision and dependable primitives; the user supplies the mathematical question and the test of whether the answer deserves belief.

Bibliography

Astropy Collaboration. *Astronomical Coordinate Systems*. Astropy Project, 2026. URL https://docs.astropy.org/en/stable/coordinates/.

Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.

Charles M. Grinstead and J. Laurie Snell. *Introduction to Probability*. American Mathematical Society, 1997. URL https://math.dartmouth.edu/~prob/prob/prob.pdf.

William H. Hayt, Jack E. Kemmerly, and Steven M. Durbin. *Engineering Circuit Analysis*. McGraw-Hill Education, 9 edition, 2019.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2 edition, 2002. doi: 10.1137/1.9780898718027.

IEEE. Ieee standard for floating-point arithmetic, 2019.

Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006. doi: 10.1007/978-0-387-40065-5.

Thomas Okken. *Free42 Manual*, version 0.7.2 edition, 2026. URL https://thomasokken.com/free42/doc/42s.pdf.

Thomas Okken and SwissMicros contributors. Free42 core for dm42, 2025. URL https://github.com/swissmicros/free42.

Gilbert Strang. *Introduction to Linear Algebra*. Wellesley–Cambridge Press, 6 edition, 2023.

SwissMicros GmbH. *DM42 Function Reference*, 2026a. URL https://technical.swissmicros.com/dm42/fat/HELP/dm42functs.htm.

SwissMicros GmbH. *DM42 User Manual*, version 3.42 edition, 2026b. URL https://technical.swissmicros.com/dm42/doc/dm42_user_manual.html.

SwissMicros GmbH. *DM42 Quick Reference: Matrix Operations*, 2026c. URL https://technical.swissmicros.com/dm42/fat/HELP/dm42help.htm.

Index

COMMAND, 3, 12
COMPLEX, 15
CROSS, 2, 9

DELR, 17, 18
DET, 2, 27, 36, 53, 54
DIM, 2, 13, 14, 64
DIM?, 4, 13–15, 18, 20, 38, 63
DIMS, 20
DOT, 2, 7, 8, 30, 53, 64

EDIT, 2, 14
EDITN, 2, 14
EIG, 53

FRNM, 2, 4, 8, 29, 44, 53, 56, 64, 65

GETM, 17, 21, 65
GROW, 14, 20

INDEX, 16, 18, 20, 65, 66
INSR, 17, 18
INVRT, 2, 27, 55

matrix division, 3

native capability limits, 3
NEWMAT, 2, 12

PUTM, 17, 64

R<>R, 17, 18
RANK, 7
RCLEL, 16
RCLIJ, 5, 16–18
RNRM, 2, 29, 31, 55
RSUM, 29

STOEL, 16, 17
STOIJ, 5, 16, 65

TRANS, 2, 7, 10, 14, 15, 30, 31, 39, 49, 62, 65
TRANS
    complex behavior, 7

UVEC, 2, 8, 10, 44, 53, 64

WRAP, 14, 20

XEQ, 64